

Using *sabinaHSBM* for link prediction and network reconstruction with “conditional_missing” method

[!NOTE]

Highlights information that users should take into account, even when skimming.

This guide shows a simple use case based on the “conditional_missing” method in *sabinaHSBM* to identify missing links (false negatives) in an incomplete network.

We use the example dataset **dat** included in the package to show key functionalities, including data preparation, link prediction, and network reconstruction.

Requirements and installation

There are **two ways** to use the package:

Option A: Docker (recommended)

Non-native UNIX users or those who prefer to avoid manual dependency installation can use the provided ready-to-use Docker image which includes R, Python, all dependencies, and *sabinaHSBM* pre-installed (see Supporting Information S1 for Docker container setup).

Once the container is running, simply load the package in your R session with:

```
library(sabinaHSBM)
```

Option B: Native UNIX installation

Native UNIX users can run *sabinaHSBM* locally **if** their system includes:

- R (version $\leq 4.0.4$) with all required R packages (listed below)
- Python $\leq$ with the **graph-tool** library (version ≤ 2.45)

```
install.packages("remotes")
# If the package is not installed, install sabinaHSBM from GitHub
if (!requireNamespace("sabinaHSBM", quietly = TRUE)) {
  remotes::install_github("h-lima/sabinaHSBM")
}

# Load sabinaHSBM
library(sabinaHSBM)
```

Install required R packages if not already available:

```
list.of.packages <- c(
  "dplyr",
  "reshape2",
  "reticulate",
  "stringr",
  "tidyr",
  "ROCR"
```

```

)

new.packages <-
  list.of.packages[!(list.of.packages %in% installed.packages()[, "Package"])]
if (length(new.packages) > 0) {
  install.packages(new.packages, dependencies = TRUE)
}

for (package in list.of.packages) {
  library(package, character.only = TRUE)
}

# Record starting time (optional)
start_time <- Sys.time()

```

Load example data

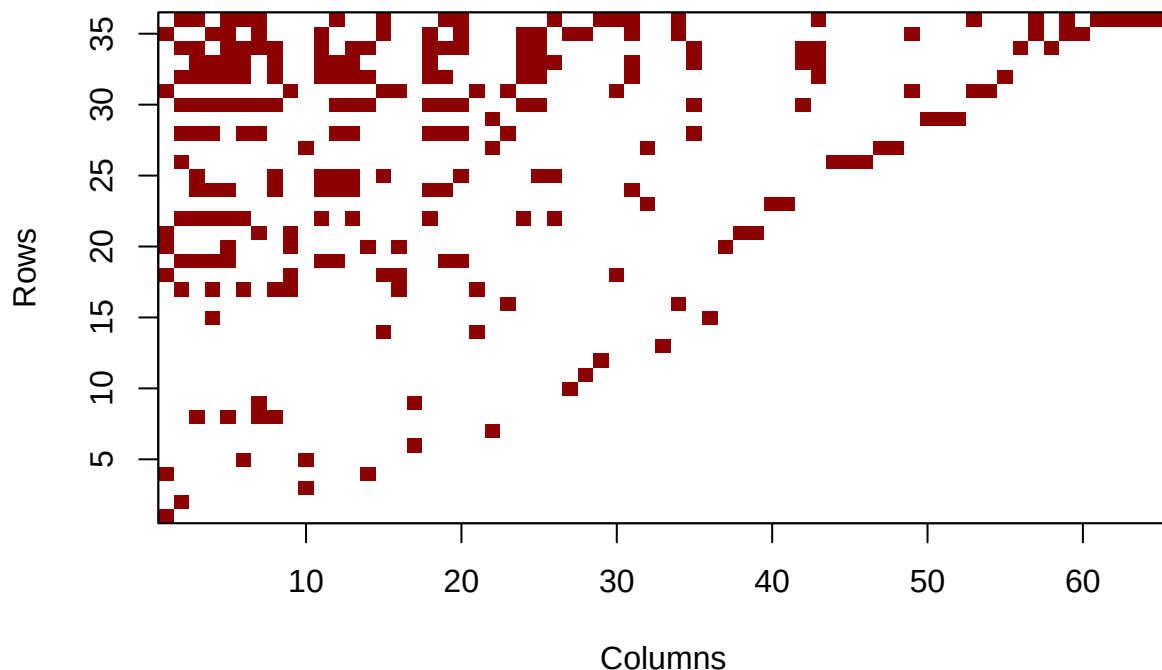
The dataset `dat` is a binary matrix representing a hypothetical species interactions network. Columns and rows correspond to two different types of nodes (e.g. hosts and parasites), and links (values of 1, in red) represent interaction between them while 0 (in black) represent lack of an observed interaction. The network contains gaps, indicating potential missing links.

```

# Load the dataset
#data(dat, package = "sabinaHSEBM")
load("../testing/dat.RData")

# Plot the simulated matrix
plot_interaction_matrix(dat)

```



Preparing Input data for HSBM

The `hsbm.input` function cleans the dataset, and creates cross-validation folds by temporarily holding out subsets of observed links. Internally, it ensures that each fold masks a representative sample. In our example, it randomly splits the 233 observed links into five similar-sized subsets (because here we use 5-fold cross-validation), holding out one subset per fold to test the model’s ability to recover missing links.

```
# Prepare input object
myInput <- hsbm.input(
  dat,           # Binary matrix of observed links
  n_folds = 10   # Number of folds for cross-validation
)
```

```
# Summarizes network characteristics
summary(myInput)
```

```
##   n_rows n_cols n_links rows_single_link cols_single_link possible_links
## 1     36    65    233                9                25          2340
```

Predicting missing links with “conditional_missing”

The `hsbm.predict` works directly with the processed input created by `hsbm.input`. It applies HSBM to each fold’s training network and computes posterior probabilities for links and group assignments for nodes. The `conditional_missing` method focuses exclusively on predicting the conditional posterior probability of unobserved links (0s) given the network’s learned block structure, useful to identify missing links (i.e., unobserved host-parasite interactions likely to exist) in partially incomplete networks—without casting doubt

on any documented interactions. Computation can be intensive, but parallelization across folds is supported with the `n_cores` argument.

```
# Predict missing links using HSBM
myPred <- hsbm.predict(
  myInput,          # Input data processed by hsbm.input()
  iter = 10000,     # Number of iterations
  wait = 1000,      # Number of iterations for MCMC equilibration
  rnd_seed = 123,   # Sets seed in python environment for reproducibility
  method = "conditional_missing", # Prediction method
  save_blocks = TRUE, # Save group assignments
  save_pickle = FALSE, # Save results as pickle files,
  save_plots = FALSE, # Save hierarchical edge bundling plots
  n_cores = 2 # Number of cores to use
)
```

Predicted probabilities and group assignments are stored for each fold. Below, we extract the probabilities (p) and groups for fold 1. The lower the p , the less likely the link exists and the lower its priority; conversely, higher p values are the most promising missing interactions.

```
# View probabilities for fold 1
probabilities_fold1 <- myPred$probs[[1]]
head(probabilities_fold1)
```

```
##   v1 v2      p v1_names v2_names  edge_type
## 1  0 38 0.01999087   row1    col3 reconstructed
## 2  0 40 0.08748079   row1    col5 reconstructed
## 3  0 41 0.03350699   row1    col6 reconstructed
## 4  0 42 0.02961120   row1    col7 reconstructed
## 5  0 43 0.04979654   row1    col8      held_out
## 6  0 44 0.01390001   row1    col9 reconstructed
```

The group assignments provide the hierarchical clustering structure of nodes for each fold. Let's extract and examine the group assignments for fold 1:

```
# View the group/block assignments for fold 1
groups_fold1 <- myPred$groups[[1]]

# Filter one type of nodes (nodes in columns)
vnames <- colnames(myPred$data)
groups_fold1 <- groups_fold1 %>% filter(name %in% vnames)
g_cols <- grep("^G", names(groups_fold1))
groups_fold1[g_cols] <- lapply(groups_fold1[g_cols], sort)

#print(head(groups_fold1))
print(lapply(myPred$groups,
  function(x){
    g_cols <- grep("^G", names(x))
    return(apply(x[g_cols], 2, function(y) length(unique(y))))
  })))
```

```
## [[1]]
## G1 G2 G3 G4
##  4  2  1  1
##
## [[2]]
## G1 G2 G3 G4
```

```
## 4 2 1 1
##
## [[3]]
## G1 G2 G3 G4
## 5 2 1 1
##
## [[4]]
## G1 G2 G3 G4
## 4 2 1 1
##
## [[5]]
## G1 G2 G3 G4
## 4 2 1 1
##
## [[6]]
## G1 G2 G3 G4
## 4 2 1 1
##
## [[7]]
## G1 G2 G3 G4
## 5 2 1 1
##
## [[8]]
## G1 G2 G3 G4
## 4 2 1 1
##
## [[9]]
## G1 G2 G3 G4
## 5 2 1 1
##
## [[10]]
## G1 G2 G3 G4
## 5 2 1 1
```

Hierarchical group assignments provide insight into how nodes (e.g., hosts) are organized across multiple levels. At the first level (G1), nodes are divided into specific groups, reflecting fine-scale patterns. Moving to higher levels (G2, G3, G4), these groups are progressively aggregated, revealing broader patterns and relationships or communities.

Network Reconstruction

The `hsbm.reconstructed` function generates a reconstructed binary interaction matrix by combining the probabilities across all folds and transforming them into a new binary (0/1) matrix/network from a user-specified threshold.

```
# Network reconstruction
myReconst <- hsbm.reconstructed(
  myPred,                      # Predictions processed by hsbm.predict
  rm_documented = TRUE,       # Use of documented entries during validation
  threshold = "prc_closest_topright", # Binarization threshold
  consistency_matrix = "average_thresholded" # Combine fold predictions
)
```

This output includes the averaged probability matrix, the final reconstructed matrix, and evaluation metrics. These results highlight the predicted interactions, showcasing the method's ability to detect missing links

effectively. Let's explore some of the key outputs in detail.

```
# View the reconstructed network summary and evaluation metrics
summary(myReconst)

## $`Reconstructed Network Metrics`
##   obs_links unobs_links pred_links kept_links spurious_links missing_links
## 1      233      2107      283      233      0      50
##
## $`Evaluation Metrics`
##   mean_RLF mean_auc mean_aucpr mean_yPRC mean_prec mean_sens mean_spec
## 1      0.62 0.8953773 0.1875282 0.009402915 0.1900492      0.62 0.959943
##   mean_ACC mean_ERR mean_tss
## 1 0.9567466 0.04325341 0.579943

# Network reconstruction
myReconst2 <- hsbm.reconstructed(
  myPred,                # Predictions processed by hsbm.predict
  rm_documented = TRUE,  # Use of documented entries during validation
  consistency_matrix = "average_thresholded" # Combine fold predictions
)

# View the reconstructed network summary and evaluation metrics
summary(myReconst)

## $`Reconstructed Network Metrics`
##   obs_links unobs_links pred_links kept_links spurious_links missing_links
## 1      233      2107      283      233      0      50
##
## $`Evaluation Metrics`
##   mean_RLF mean_auc mean_aucpr mean_yPRC mean_prec mean_sens mean_spec
## 1      0.62 0.8953773 0.1875282 0.009402915 0.1900492      0.62 0.959943
##   mean_ACC mean_ERR mean_tss
## 1 0.9567466 0.04325341 0.579943
```

The summary provides the number of missing links, as well as key evaluation metrics, such as the recovery link fraction (RLF).

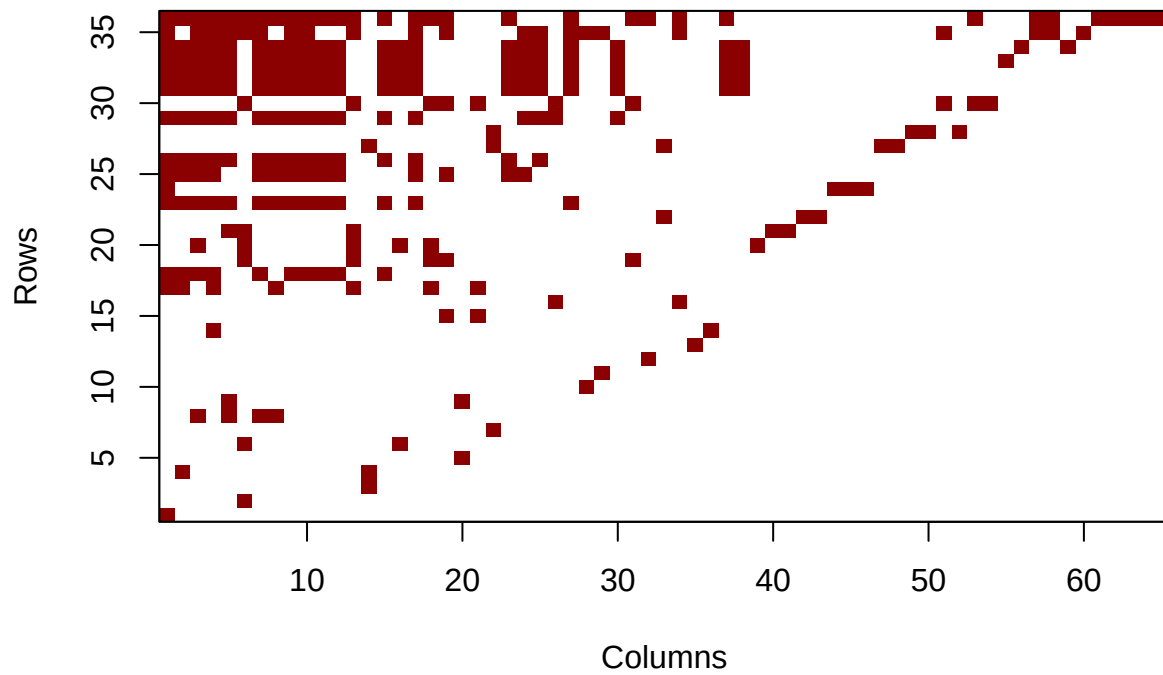
```
# View the top potential missing links
top_links_df <- top_links(myReconst,
  n = 10,
  edge_type = "undocumented") # Type of edge to rank

print(top_links_df)
```

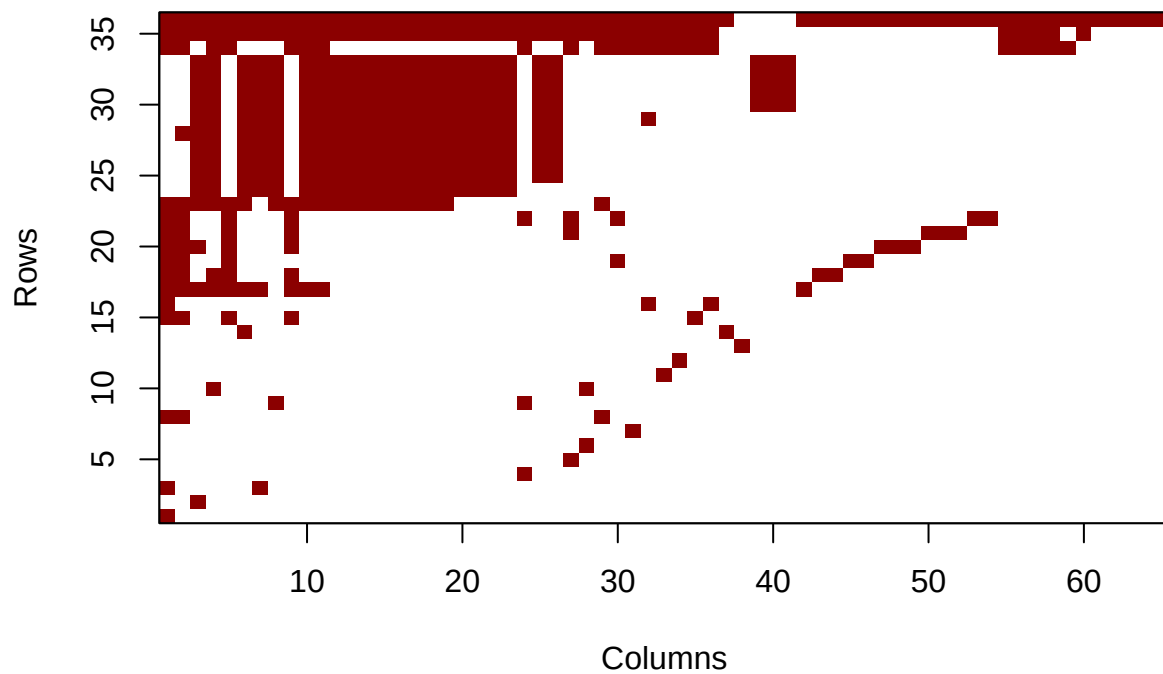
```
##   v1_names v2_names      p      sd
## 1    row4    col8 0.4709448 0.05794638
## 2    row4   col12 0.3807215 0.10193875
## 3    row7   col10 0.3357914 0.07102567
## 4    row5   col16 0.2836251 0.06443609
## 5    row6   col11 0.2349314 0.06687872
## 6    row3   col19 0.2228809 0.04903951
## 7    row2  col24 0.2125150 0.06870871
## 8    row7    col3 0.2081772 0.04835714
## 9    row3    col1 0.2039345 0.08595254
## 10   row4  col22 0.2015019 0.06285083
```

The `top-links` function identifies the undocumented links most likely to be missing links in a network predicted by HSBM.

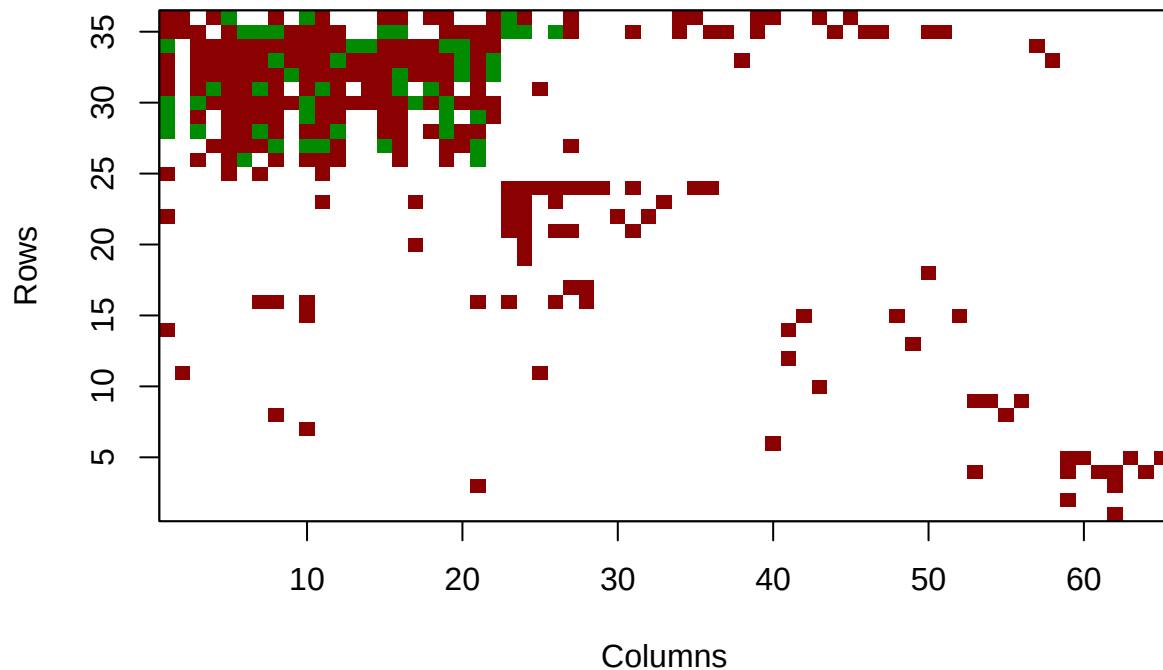
```
# View the reconstructed binary matrix  
plot_interaction_matrix(myReconst$new_mat)
```



```
plot_interaction_matrix(myReconst2$new_mat)
```



```
# Plot the matrix with spurious
cols <- c("white", "red4")
dat_spur <- myReconst$data
new_missing <- which(myReconst$data == 0 & myReconst$new_mat == 1)
if(length(new_missing)!=0){
  dat_spur[new_missing] <- 2
  cols <- c(cols, "green4")
}
plot_interaction_matrix(dat_spur,
  col = cols, order_mat = FALSE)
```

This document demonstrates the use of the *sabinaHSBM* package for network reconstruction. By applying the `conditional_missing` method, we showcased how missing links can be effectively identified and addressed in incomplete networks.

Computing characteristics

```
# Show processing time and computer characteristics (optional)
end_time <- Sys.time()
cat("The processing time of this script took: ",
    round(as.numeric(difftime(end_time, start_time, units = "mins", 1))), "minutes\n")
```

```
## The processing time of this script took: 33 minutes
```

This analysis was performed on a Dell Inspiron notebook with the following characteristics:

- Processor (CPU): Intel Core i7-7500U @ 3.5GHz
- Memory (RAM): 16 GB
- Operating System: Ubuntu 24.04 (run on a docker image)
- R Version: 4.3.3